

Logic Coverage Criteria: Applied to test code

Meenakshi D'Souza

International Institute of Information Technology
Bangalore.

September 13, 2021

Logical predicates from source code

- Predicates are derived from decision statements.
  - if-then-else, switch-case, while-do, do-while, for etc.
- In programs, most predicates have less than four clauses.
- When a predicate only has one clause, COC, ACC, ICC, and CC all collapse to predicate coverage (PC).

NPTEL

RIPR criteria for source code

Applying logic criteria to program source is hard because of reachability and controllability.

- Reachability: Before applying the criteria on a predicate at a particular statement, we have to get to that statement.
- Controllability: We have to find input values that indirectly assign values to the variables in the predicates.
  - Variables in the predicates that are not inputs to the program are called internal variables.
- Extra predicates correspondng to reachability and controllability need to be *appended* to the concerned predicate to test it.
- These concepts are illustrated using two examples.

NPTEL

Two examples

We will look at two examples to understand reachability for logical predicates involving internal variables.

- Program for a thermostat
- Program for detecting the type of a triangle

NPTEL

Thermostat program

```
1  // By Jeff Offutt & Paul Ammann // Programmable Thermostat
6  import java.io.*;
...
10 public class Thermostat
11 {
12    private int curTemp; // Current temperature reading
13    private int thresholdDiff; // Temp difference until heater on
14    private int timeSinceLastRun; // Time since heater stopped
15    private int minLag; // How long I need to wait
16    private boolean Override; // Has user overridden the program
17    private int overTemp; // OverridingTemp
18    private int runTime; //output of turnHeaterOn-how long to run
19    private boolean heaterOn; //output of turnHeaterOn-whether to run
20    private Period period; // morning, day, evening, or night
21    private DayType day; // week day or weekend day
23    // Decide whether to turn the heater on, and for how long.
```

Thermostat program

```
24 public boolean turnHeaterOn(ProgrammedSettings pSet)
25 {
26     int dTemp = pSet.getSetting(period, day);
28     if (((curTemp < dTemp - thresholdDiff) ||
29          (Override && curTemp < overTemp - thresholdDiff)) &&
30          (timeSinceLastRun > minLag))
31     {  // Turn on the heater
32        // How long? Assume 1 minute per degree (Fahrenheit)
33        int timeNeeded = curTemp - dTemp;
34        if (Override)
35           timeNeeded = curTemp - overTemp;
36        setRunTime(timeNeeded);
37        setHeaterOn(true);
38        return(true);
39     }
40     else
41     {
42        setHeaterOn(false);
43        return(false);
44     } } // End turnHeaterOn
```

NPTEL

Two Thermostat predicates

- Lines 28-30: `(((curTemp < dTemp - thresholdDiff) || (Override && curTemp < overTemp - thresholdDiff)) && timeSinceLastRun > minLag)`
- Line 34: `(Override)`
- We simplify as follows: Let
  - a: `curTemp < dTemp - thresholdDiff`
  - b: `Override`
  - c: `curTemp < overTemp - thresholdDiff`
  - d: `timeSinceLastRun > minLag`
- Now, we have $p = (a||(b\&\&c))\&\&d$ and 34: b.

Reachability for `Thermostat` predicates

- The clause *a* has an internal variable `dTemp`.
- *Solving* for `dTemp`: Line 26 says
  `dTemp = pSet.getSetting (period, day);`
- We solve as follows:

```
 getSetting(Period.MORNING, DayType.WEEKDAY, 69);
setPeriod(Period.MORNING);
setDay(DayType.WEEKDAY);
```

NPTEL

Predicate coverage: p being true

- a: `curTemp < dTemp - thresholdDiff`: true
- b: `Override`: true
- c: `curTemp < overTemp - thresholdDiff`: true
- d: `timeSinceLastRun > (minLag)`: true

NPTEL

Predicate coverage: *p* being true: Test cases

- thermo = new Thermostat(); // Needed object
- settings = new ProgrammedSettings(); // Needed object
- settings.setSetting (Period.MORNING, DayType.WEEKDAY, 69); // dTemp
- thermo.setPeriod (Period.MORNING); // dTemp
- thermo.setDay (DayType.WEEKDAY); // dTemp
- thermo.setCurrentTemp (63); // clause *a*
- thermo.setThresholdDiff (5); // clause *a*
- thermo.setOverride (true); // clause *b*
- thermo.setOverTemp (70); // clause *c*
- thermo.setMinLag (10); // clause *d*
- thermo.setTimeSinceLastRun (12); // clause *d*
- assertTrue (thermo.turnHeaterOn (settings)); // Run test

Towards CACC for p

- We illustrate the steps for CACC coverage for a being the major clause for $p = (a||(b\&\&c))\&\&d$.
- Determining p_a:

$$\begin{aligned} p_a &= p_{a=true} \oplus p_{a=false} \\ &= (true||(b\&\&c)\&\&d) \oplus (false||(b\&\&c))\&\&d) \\ &= (true\&\&d) \oplus ((b\&\&c)\&\&d) \\ &= d \oplus ((b\&\&c)\&\&d) \\ &= !(b\&\&c)\&\&d \\ &= (!b||!c)\&\&d \end{aligned}$$

- Similarly, we can determine p_b, p_c and p_d.

NPTEL

CACC for p

TR for CACC for p for each clause to be major clause:

	a	b	c	d
p_a:	T	t	f	t
	F	t	f	t
p_b:	f	T	t	t
	f	F	t	t
p_c:	f	t	T	t
	f	t	F	t
p_d:	t	t	t	T
	t	t	t	F

Rows 3 and 5 are duplicates, six tests are needed to achieve CACC TR for p.

Test case inputs for CACC for p

- For a:
  - $a = t$: curTemp $<$ dTemp-thresholdDiff, curTemp=63, dTemp=69, thresholdDiff=5
  - $a = f$: !(curTemp $<$ dTemp-thresholdDiff), curTemp=66, dTemp= 69, thresholdDiff=5
- dTemp
  settings.setSettings (Period.MORNING, DayType.WEEKDAY, 69)
  thermo.setPeriod (Period.MORNING);
  thermo.setDay (Daytype.WEEKDAY);
- For b:
  - $b = t$: Override=T
  - $b = f$: !Override=F

Test case inputs for CACC for p, contd.

- For c:
  - $c = t$: curTemp $<$ overTemp-thresholdDiff, curTemp=63, dTemp=72, thresholdDiff=5
  - $c = f$: !(curTemp $<$ overTemp-thresholdDiff), curTemp=66, dTemp= 67, thresholdDiff=5
- For d:
  - $d = t$: timeSinceLastRun $>$ minLag timeSinceLastRun=12, minLag=10
  - $d = f$: !(timeSinceLastRun $<$ minLag) timeSinceLastRun=8, minLag=10

Test cases for CACC, #1

dTemp = 69 (period = MORNING, daytype = WEEKDAY)

- 1. T t f t

  thermo.setCurrentTemp (63);
  thermo.setThresholdDiff (5);
  thermo.setOverride (true);
  thermo.setOverTemp (67); // c is false
  thermo.setMinLag (10);
  thermo.setTimeSinceLastRun (12);

- 2. F t f t

  thermo.setCurrentTemp (66); // a is false
  thermo.setThresholdDiff (5);
  thermo.setOverride (true);
  thermo.setOverTemp (67); // c is false
  thermo.setMinLag (10);
  thermo.setTimeSinceLastRun (12);

Test cases for CACC, #1

dTemp = 69 (period = MORNING, daytype = WEEKDAY)

- 3. f T t t
  thermo.setCurrentTemp (66); // a is false
  thermo.setThresholdDiff (5);
  thermo.setOverride (true);
  thermo.setOverTemp (72); // to make c true
  thermo.setMinLag (10);
  thermo.setTimeSinceLastRun (12);
- 4. F f T t thermo.setCurrentTemp (66); // a is false
  thermo.setThresholdDiff (5);
  thermo.setOverride (false); // b is false
  thermo.setOverTemp (72);
  thermo.setMinLag (10);
  thermo.setTimeSinceLastRun (12);

Test cases for CACC, #1

dTemp = 69 (period = MORNING, daytype = WEEKDAY)

- 5. t t t T
  thermo.setCurrentTemp (63);
  thermo.setThresholdDiff (5);
  thermo.setOverride (true);
  thermo.setOverTemp (72);
  thermo.setMinLag (10);
  thermo.setTimeSinceLastRun (12);
- 6. t t t F
  thermo.setCurrentTemp (63);
  thermo.setThresholdDiff (5);
  thermo.setOverride (true);
  thermo.setOverTemp (72);
  thermo.setMinLag (10);
  thermo.setTimeSinceLastRun (8); // d is false

Logic coverage over source code

Our exercise of doing logic coverage criteria for predicate coverage and CACC for Thermostat example is complete!

NPTEL

Credits

Part of the material used in these slides is derived from the presentations of the book Introduction to Software Testing, by Paul Ammann and Jeff Offutt.

NPTEL